



Lecture # 5 Basic SQL

Rashmi Dutta Baruah

Department of Computer Science & Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati

Guwahati - 781039, INDIA



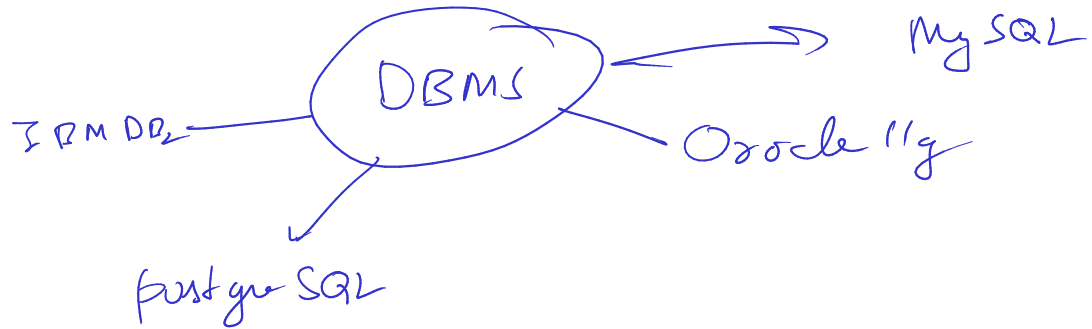


Outline

- SQL Data definition and Data types
- Specifying Constraints in SQL
- Basic retrieval queries in SQL
- Insert, Delete, and Update Statements in SQL

SQL:

- Structured Query Language
- IBM, system R
- Include features from Relational Algebra, Tuple Relational Calculus
- DBMS Independent.



→ DDL: Data definition language

→ DML: Data manipulation language

<u>Relational Algebra</u>	<u>SQL</u>
Relation	Table
Tuple	Row
Attribute	Column



SQL Data definition and Data types

- CREATE SCHEMA ; *mysql> CREATE DATABASE company;*
mysql> USE company;

- Specifies a new database schema by giving it a name

CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith'

- CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database

SQL> CREATE TABLE [schema]. Table name (
[column desc.],);



CREATE TABLE

- Specifies a new **base relation (or base table)** by giving it a name, and specifying each of its attributes and their data types (INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n)).
- A constraint NOT NULL may be specified on an attribute

```
✓ CREATE TABLE    DEPARTMENT
    (      DNAME VARCHAR(15)  NOT NULL,
          DNUMBER      INTEGER      NOT NULL,
          MGRSSN CHAR(9) ,
          MGRSTARTDATE DATE    );
```

INT
INT(6)

- Data type can be specified directly, or alternatively a domain can be declared and the domain name can be used with the attribute specification.

```
CREATE DOMAIN SSN_TYPE AS CHAR(9)
```

```
mysql> CREATE DOMAIN Roll-no_Type AS INT(6);
```

Data Types:

→ Numerics:

INT, SMALLINT, FLOAT, REAL, DOUBLE PRECISION

→ Formatted Numbers:

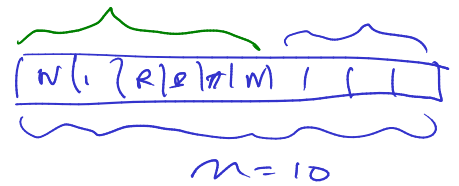
DECIMAL (i, j) : ~~eg~~ DECIMAL (3, 2)
↳ 578.25

NUMERIC (i, j)

→ Character Strings:

→ Fixed length: CHAR (n) :

→ Varying length: VARCHAR (n)



→ Bit Strings:

BIT (j)

→ DATE & Time:

DATE, TIME, TIMESTAMP

→ Images:

BLOB

→ Text:

TEXT

CREATE TABLE

- CREATE TABLE command for specifying the primary key attributes, secondary keys, and referential integrity constraints (foreign keys).
- Key attributes can be specified via the PRIMARY KEY and UNIQUE phrases

DNAME	DNUMBER	MGRSSN
NOT NULL		

CREATE TABLE DEPT

(DNAME VARCHAR(10)

DNUMBER INTEGER

MGRSSN CHAR(9),

MGRSTARTDATE DATE,

✓ **PRIMARY KEY** (DNUMBER), ←

✓ **UNIQUE** (DNAME),

✓ **FOREIGN KEY** (MGRSSN) REFERENCES EMP (Ssn));

SSN	NAME
101	Row
102	
103	

DNAME	DNUMBER	MGRSSN
201	CSE	101



```
mysql> DROP TABLE EMP;
```

DROP TABLE

, DATABASE, SCHEMA

- ✓ Used to remove a relation (base table) and its definition
- The relation can no longer be used in queries, updates, or any other commands since its description no longer exists
- Example:

```
DROP TABLE DEPENDENT;
```





ALTER TABLE

- Used to add an attribute to one of the base relations
- The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute
- Example:

ALTER TABLE EMPLOYEE ADD JOB VARCHAR(12);

- The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple. This can be done using the UPDATE command.

Specifying Constraints

- Attribute constraints and attribute defaults
 - NOT NULL constraint
 - to define a default value for an attribute, append the clause DEFAULT <value> to an attribute definition

```
CREATE TABLE DEPT
(
  ...
  MGRSSN CHAR(9) NOT NULL DEFAULT '888665555'
  ...);
```

- CHECK clause can be used following an attribute or domain definition to restrict the attribute or domain values

→ DNUMBER INT NOT NULL CHECK (Dnumber > 0 and Dnumber < 21)

(0, 21)
D_NUM

SQL> CREATE DOMAIN D_NUM AS INTEGER
CHECK (D_NUM > 0 AND D_NUM < 21)



REFERENTIAL INTEGRITY OPTIONS

- We can specify RESTRICT, CASCADE, SET NULL or SET DEFAULT on referential integrity constraints (foreign keys)

```
CREATE TABLE DEPT
( DNAME VARCHAR(10) NOT NULL,
  DNUMBER INTEGER NOT NULL,
  MGRSSN CHAR(9),
  MGRSTARTDATE CHAR(9),
  PRIMARY KEY (DNUMBER),
  UNIQUE (DNAME),
  FOREIGN KEY (MGRSSN) REFERENCES EMP (Ssn)
ON DELETE SET NULL ON UPDATE CASCADE );
```



REFERENTIAL INTEGRITY OPTIONS (continued)

```
CREATE TABLE EMP
(
    ENAME VARCHAR(30) NOT NULL,
    ESSN CHAR(9),
    BDATE DATE,
    — DNO INTEGER DEFAULT 1,
    SUPERSSN CHAR(9),
    PRIMARY KEY (ESSN),
    FOREIGN KEY (DNO) REFERENCES DEPT
        ON DELETE SET DEFAULT ON UPDATE CASCADE,
    FOREIGN KEY (SUPERSSN) REFERENCES EMP (Snn)
        ON DELETE SET NULL ON UPDATE CASCADE );
```



✓ Basic Retrieval Queries in SQL

- SQL has one basic statement for retrieving information from a database; the SELECT statement
- This is ***not the same*** as the SELECT operation of the relational algebra 
- Important distinction between SQL and the formal relational model; SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values
- Hence, an SQL relation (table) is a ***multi-set*** (sometimes called a bag) of tuples; it is not a set of tuples
- SQL relations can be constrained to be sets by specifying PRIMARY KEY or UNIQUE attributes, or by using the DISTINCT option in a query



Retrieval Queries in SQL (cont.)

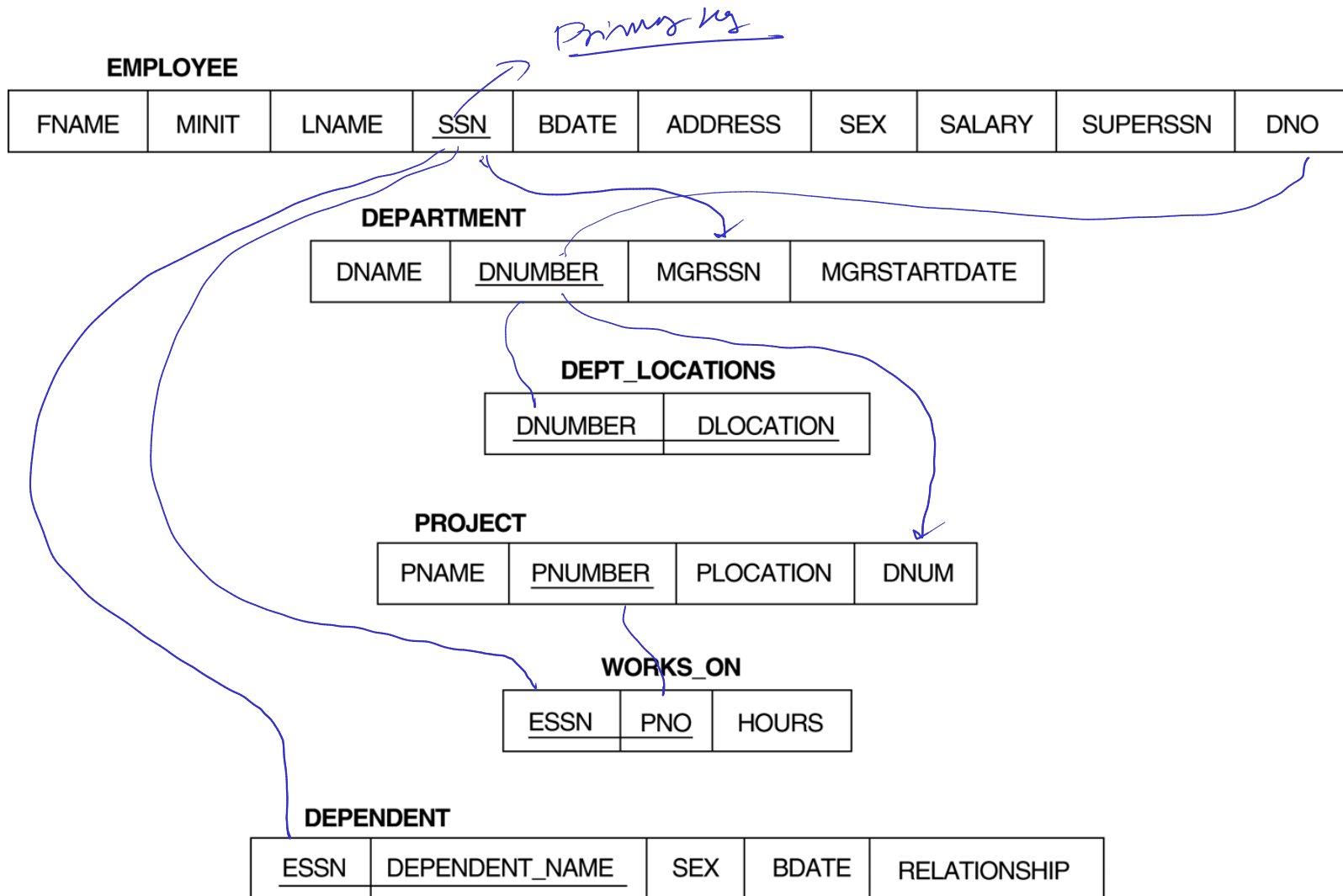
- Basic form of the SQL SELECT statement is called a *mapping* or a *SELECT-FROM-WHERE block*

✱

SQL> **SELECT** <attribute list>
FROM <table list>
WHERE <condition> Role no = 200101 ... **AND** DEPT name = 'CS2'

- <attribute list> is a list of attribute names whose values are to be retrieved by the query
- <table list> is a list of the relation names required to process the query
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Relational Database Schema



EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Populated Database



BASIC SQL Queries (cont.)

- Basic SQL queries correspond to using the SELECT, PROJECT, and JOIN operations of the relational algebra
- All subsequent examples use the COMPANY database
- Example of a simple query on *one* relation
- Query 0: Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.

```
Q0 : SELECT      BDATE, ADDRESS
      FROM        EMPLOYEE
      WHERE  FNAME='John' AND MINIT='B'
      AND        LNAME='Smith'
```

- Similar to a SELECT-PROJECT pair of relational algebra operations; the SELECT-clause specifies the *projection attributes* and the WHERE-clause specifies the *selection condition*
- However, the result of the query *may contain* duplicate tuples



Basic SQL Queries (cont.)

- Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1 : SELECT FNAME, LNAME, ADDRESS  
      FROM   EMPLOYEE, DEPARTMENT  
      WHERE  DNAME='Research' AND DNUMBER=DNO
```

- Similar to a SELECT-PROJECT-JOIN sequence of relational algebra operations
- (DNAME='Research') is a *selection condition* (corresponds to a SELECT operation in relational algebra)
- (DNUMBER=DNO) is a *join condition* (corresponds to a JOIN operation in relational algebra)



Simple SQL Queries (cont.)

- Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
Q2 : SELECT PNUMBER, DNUM, LNAME, BDATE, ADDRESS  
  
      FROM   PROJECT, DEPARTMENT, EMPLOYEE  
  
      WHERE  DNUM=DNUMBER AND MGRSSN=SSN AND  
             PLOCATION='Stafford'
```

- In Q2, there are *two* join conditions
- The join condition DNUM=DNUMBER relates a project to its controlling department
- The join condition MGRSSN=SSN relates the controlling department to the employee who manages that department



Aliases, * and DISTINCT, Empty WHERE-clause

- In SQL, we can use the same name for two (or more) attributes as long as the attributes are in *different relations*

A query that refers to two or more attributes with the same name must *qualify* the attribute name with the relation name by *prefixing* the relation name to the attribute name

Example:

- EMPLOYEE.LNAME, DEPARTMENT.DNAME



ALIASES

- Some queries need to refer to the same relation twice
- In this case, *aliases* are given to the relation name
- Query 8: For each employee, retrieve the employee's name, and the name of his or her immediate supervisor.

```
Q8 :  SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME  
      FROM   EMPLOYEE E S  
      WHERE  E.SUPERSSN=S.SSN
```

- In Q8, the alternate relation names E and S are called *aliases* or *tuple variables* for the EMPLOYEE relation
- We can think of E and S as two *different copies* of EMPLOYEE; E represents employees in role of *supervisees* and S represents employees in role of *supervisors*



ALIASES (cont.)

- Aliasing can also be used in any SQL query for convenience
Can also use the **AS** keyword to specify aliases

Q8: SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
 FROM EMPLOYEE AS E, EMPLOYEE AS S
 WHERE E.SUPERSSN=S.SSN



UNSPECIFIED WHERE-clause

- A missing *WHERE-clause* indicates no condition; hence, *all tuples* of the relations in the FROM-clause are selected
- This is equivalent to the condition WHERE TRUE
- Query 9: Retrieve the SSN values for all employees.

```
Q9: SELECT SSN  
      FROM EMPLOYEE
```

- If more than one relation is specified in the FROM-clause *and* there is no join condition, then the *CARTESIAN PRODUCT* of tuples is selected



UNSPECIFIED WHERE-clause (cont.)

- Example:

Q10: SELECT SSN, DNAME
 FROM EMPLOYEE, DEPARTMENT

- It is extremely important not to overlook specifying any selection and join conditions in the WHERE-clause; otherwise, incorrect and very large relations may result



USE OF *

- To retrieve all the attribute values of the selected tuples, a * is used, which stands for *all the attributes*

Examples:

Q1C: SELECT *
 FROM EMPLOYEE
 WHERE DNO=5

Q1D: SELECT *
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNAME='Research' AND
 DNO=DNUMBER



USE OF DISTINCT

- SQL does not treat a relation as a set; *duplicate tuples can appear*
- To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used
- For example, the result of Q11 may have duplicate SALARY values whereas Q11A does not have any duplicate values

Q11: **SELECT SALARY**
 FROM EMPLOYEE

Q11A: **SELECT DISTINCT SALARY**
 FROM EMPLOYEE



SET OPERATIONS

- SQL has directly incorporated some set operations
- There is a union operation (**UNION**), and in *some versions* of SQL there are set difference (**MINUS**) and intersection (**INTERSECT**) operations
- The resulting relations of these set operations are sets of tuples; *duplicate tuples are eliminated from the result*
- The set operations apply only to *union compatible relations* ; the two relations must have the same attributes and the attributes must appear in the same order



SET OPERATIONS (cont.)

- Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

**Q4: (SELECT PNAME
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE DNUM=DNUMBER AND MGRSSN=SSN AND
LNAME='Smith')
UNION (SELECT PNAME
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE PNUMBER=PNO AND ESSN=SSN AND
LNAME='Smith')**



SUBSTRING COMPARISON

- The **LIKE** comparison operator is used to compare partial strings
- Two reserved characters are used: '%' (or '*' in some implementations) replaces an arbitrary number of characters, and '_' replaces a single arbitrary character



SUBSTRING COMPARISON (cont.)

- Query 25: Retrieve all employees whose address is in Houston, Texas. Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX'.

Q25:	SELECT	FNAME, LNAME
	FROM	EMPLOYEE
	WHERE	ADDRESS LIKE
		'%Houston,TX%'



SUBSTRING COMPARISON (cont.)

- Query 26: Retrieve all employees who were born during the 1950s.

-

Q26: **SELECT FNAME, LNAME**
 FROM EMPLOYEE
 WHERE BDATE LIKE '___5_____'

- The LIKE operator allows us to get around the fact that each value is considered atomic and indivisible; hence, in SQL, character string attribute values are not atomic



ARITHMETIC OPERATIONS

- The standard arithmetic operators '+', '-', '*', and '/' (for addition, subtraction, multiplication, and division, respectively) can be applied to numeric values in an SQL query result
- Query 27: Show the effect of giving all employees who work on the 'ProductX' project a 10% raise.

```
Q27: SELECT  FNAME, LNAME, 1.1*SALARY  
          FROM    EMPLOYEE, WORKS_ON, PROJECT  
          WHERE   SSN=ESSN AND PNO=PNUMBER AND  
                PNAME='ProductX'
```



ORDER BY

- The **ORDER BY** clause is used to sort the tuples in a query result based on the values of some attribute(s)
- Query 28: Retrieve a list of employees and the projects each works in, ordered by the employee's department, and within each department ordered alphabetically by employee last name.

```
Q28: SELECT  DNAME, LNAME, FNAME, PNAME
           FROM      DEPARTMENT, EMPLOYEE,
                   WORKS_ON, PROJECT
           WHERE  DNUMBER=DNO AND SSN=ESSN           AND
                   PNO=PNUMBER
           ORDER BY      DNAME, LNAME
```



ORDER BY (cont.)

- The default order is in ascending order of values
- We can specify the keyword **DESC** if we want a descending order; the keyword **ASC** can be used to explicitly specify ascending order, even though it is the default



Specifying Updates in SQL

- There are three SQL commands to modify the database; INSERT, DELETE, and UPDATE



INSERT

- In its simplest form, it is used to add one or more tuples to a relation
- Attribute values should be listed in the same order as the attributes were specified in the CREATE TABLE command



INSERT (cont.)

- Example:

**U1: INSERT INTO EMPLOYEE
VALUES ('Richard','K','Marini', '653298653', '30-DEC-52',
'98 Oak Forest,Katy,TX', 'M', 37000,'987654321', 4)**

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple
- Attributes with NULL values can be left out
- Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.

**U1A: INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
VALUES ('Richard', 'Marini', '653298653')**



INSERT (cont.)

- Important Note: Only the constraints specified in the DDL commands are automatically enforced by the DBMS when updates are applied to the database
- Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation



INSERT (cont.)

- Example: Suppose we want to create a temporary table that has the employee last name, project name, and hours per week for each employee working on a project.

U3A: **CREATE TABLE WORKS_ON_INFO**
 (EMP_NAME VARCHAR(15),
 PROJ_NAME VARCHAR(15),
 HOURS_PER_WEEK DECIMAL(3,1));

U3B: **INSERT INTO WORKS_ON_INFO (EMP_NAME,**
 PROJ_NAME, HOURS_PER_WEEK)
SELECT E.LANME, P.PNAME,W.HOURS
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE P,PNUMBER= W.PNO AND W.ESSN = E.SSN



INSERT (cont.)

- Note: The DEPTS_INFO table may not be up-to-date if we change the tuples in either the DEPARTMENT or the EMPLOYEE relations *after* issuing U3B. We have to create a view (see later) to keep such a table up to date.



DELETE

- Removes tuples from a relation
- Includes a WHERE-clause to select the tuples to be deleted
- Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
- A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
- The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause
- Referential integrity should be enforced



DELETE (cont.)

- Examples:

U4A: DELETE FROM EMPLOYEE
WHERE LNAME='Brown'

U4B: DELETE FROM EMPLOYEE
WHERE SSN='123456789'

U4C: DELETE FROM EMPLOYEE
WHERE DNO = 5

U4D: DELETE FROM EMPLOYEE



UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity should be enforced



UPDATE (cont.)

- Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.

**U5: UPDATE
SET
WHERE**

**PROJECT
PLOCATION = 'Bellaire', DNUM = 5
PNUMBER=10**



UPDATE (cont.)

- Example: Give all employees in the 'Research' department a 10% raise in salary.

U6: UPDATE EMPLOYEE

SET SALARY = SALARY *1.1

WHERE Dno = 5

- In this request, the modified SALARY value depends on the original SALARY value in each tuple
- The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification
- The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification



Summary

- We discussed SQL database language. This language and its variations have been implanted as interfaces to many commercial relational DBMSs.
- The following features of SQL are discussed:
 - Data definition commands for creating tables
 - SQL basic data types
 - Commands for constraint specification
 - Simple retrieval queries and database update commands